Linear Classifiers and Logistic Regression

4

Post

CS 3244
Machine Learning

NUS National University of Singapore | Computing

Recap and Forecast

Naïve Bayes:

- Naïve: assumes independence between input features
- Bayes: estimates posterior from prior, marginals and likelihoods
- Smoothing caters for unseen data

k NN:

- Instance-based non-parametric
- Votes from nearest neighbor
- Larger k to smooth decision boundaries
- Problems with high dimensional data

Other important concepts for ML:

- Smoothing, Cost (Loss) Functions, Noisy Targets and the Curse of Dimensionality

Learning Outcomes for this week

- Describe the basic idea of linear classification;
- Investigate linear classification using the Perceptron learning algorithm (PLA);
- Understand how both linear and logistic regression works;
- Understand the mathematical proof on how PLA converges;

- Other important concepts:
  Gradient Descent and Non-linear mappings.

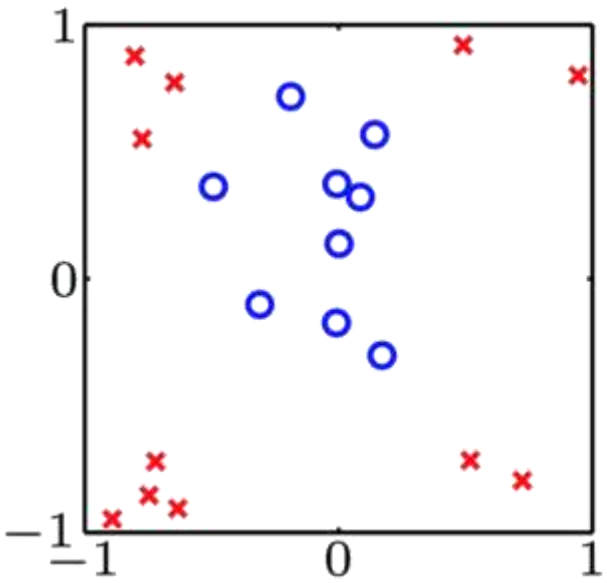
Nonlinear Mapping

CS3244 Machine Learning

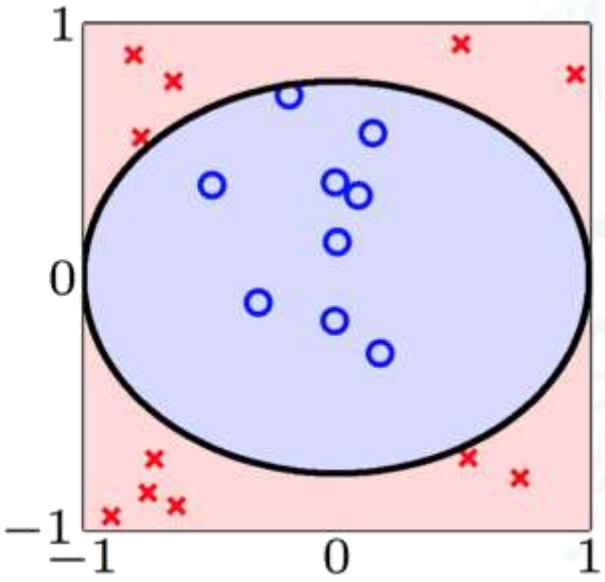
NUS National University of Singapore
Department of Computer Science
School of Computing

Linear models are limited

Data:



Hypothesis:



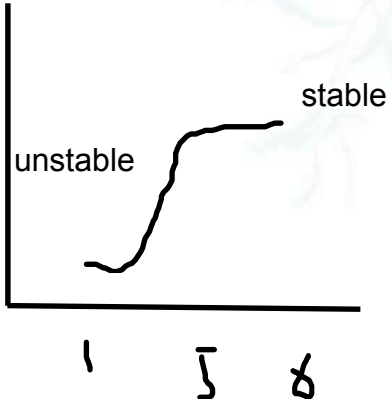
Another example

Credit line is affected by years in current residence x_i, but not in a linear way.

The value range $[\![1 < x_i < 5]\!]$ is more significant.

$[\![x]\!]$ *here means return 1 if x is true, 0 otherwise*

Can we do that with linear models?



But linear *in what?*

Linear regression implements

$$\sum_{i=0}^{n} \theta_i x_i = \theta_0 + \theta_1 x_1 + \cdots + \theta_n x_n$$

Linear classification implements

$$\text{sign}\left(\sum_{i=0}^{n} \theta_i x_i\right) \begin{cases} +1 \\ -1 \end{cases}$$

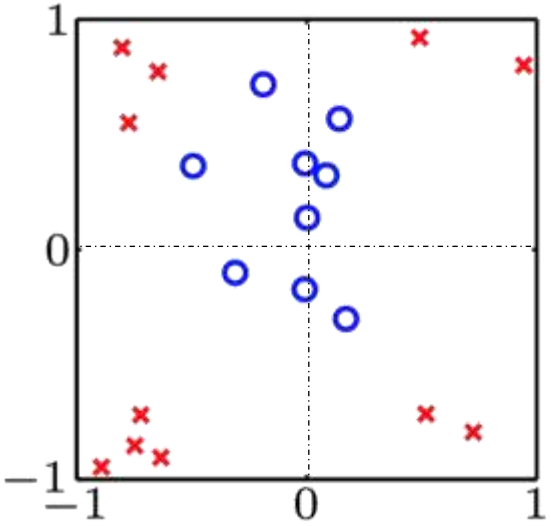
Algorithms work because of the linearity of weights,

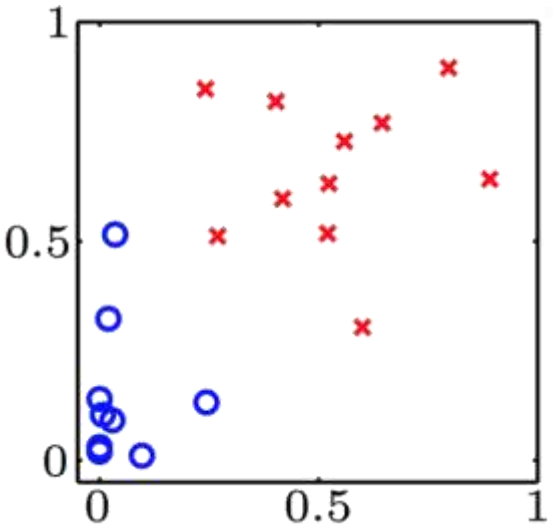
but it doesn't say anything about the observed data x.

Transform the data nonlinearly

Transformation function

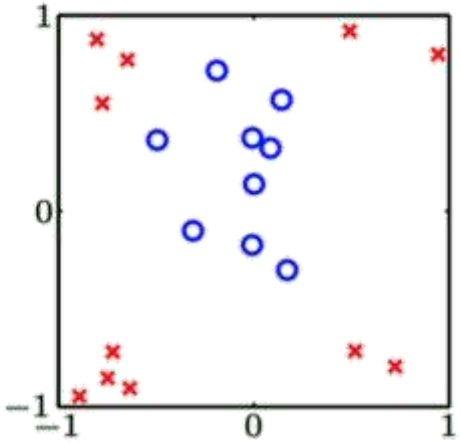
$$(x_1, x_2, \dots, x_n) \xrightarrow{\Phi} (x_1^2, x_2^2, \dots, x_n^2)$$



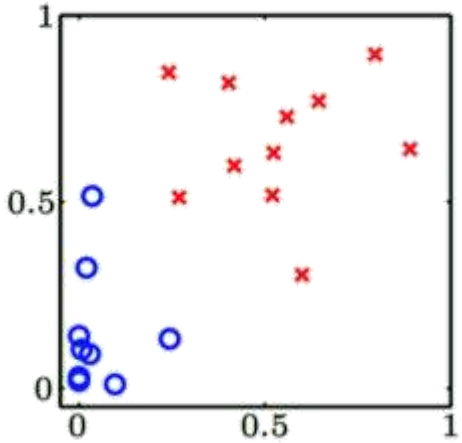


Any $\mathbf{x} \xrightarrow{\Phi} \mathbf{z}$ preserves this linearity!

1. Original Data
$\mathbf{x}^{(j)} \in \mathcal{X}$

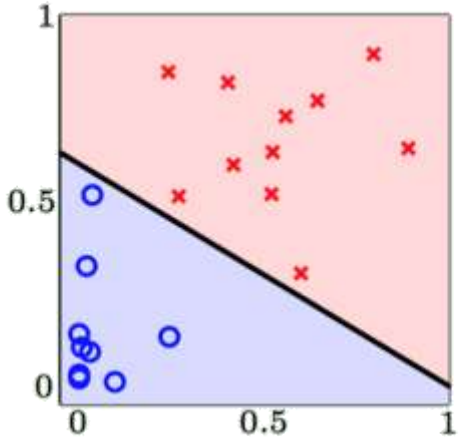


$\xrightarrow{\Phi}$



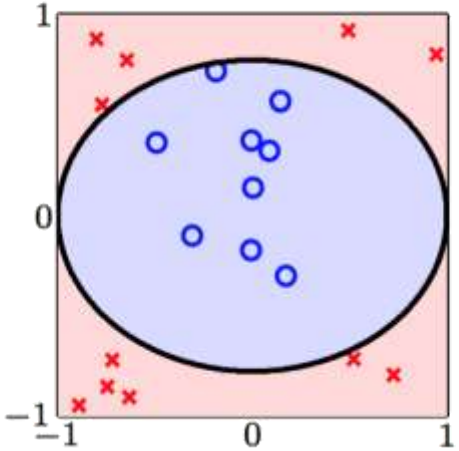
2. Transform the data
$\mathbf{z}^{(j)} = \Phi\left(\mathbf{x}^{(j)}\right) \in \mathcal{Z}$

$\downarrow$



3. Separate the data in $\mathcal{Z}$ space
$\tilde{h}(\mathbf{z}) = \mathrm{sign}\left(\tilde{\theta}^\top \mathbf{z}\right)$

$\xleftarrow{\text{'}\Phi^{-1}\text{'}}$



4. Classify in $\boldsymbol{X}$ space
$h_{\tilde{\theta}}(\mathbf{x}) = \tilde{h}\,(\Phi(\mathbf{x})$
$= \mathrm{sign}\left(\tilde{\theta}^\top \Phi(\mathbf{x})\right)$

What transforms to what

$$\mathbf{x} = (x_1, x_2, \dots, x_n) \xrightarrow{\Phi} \mathbf{z} = (z_0, z_1, \dots, z_{\tilde{n}})$$

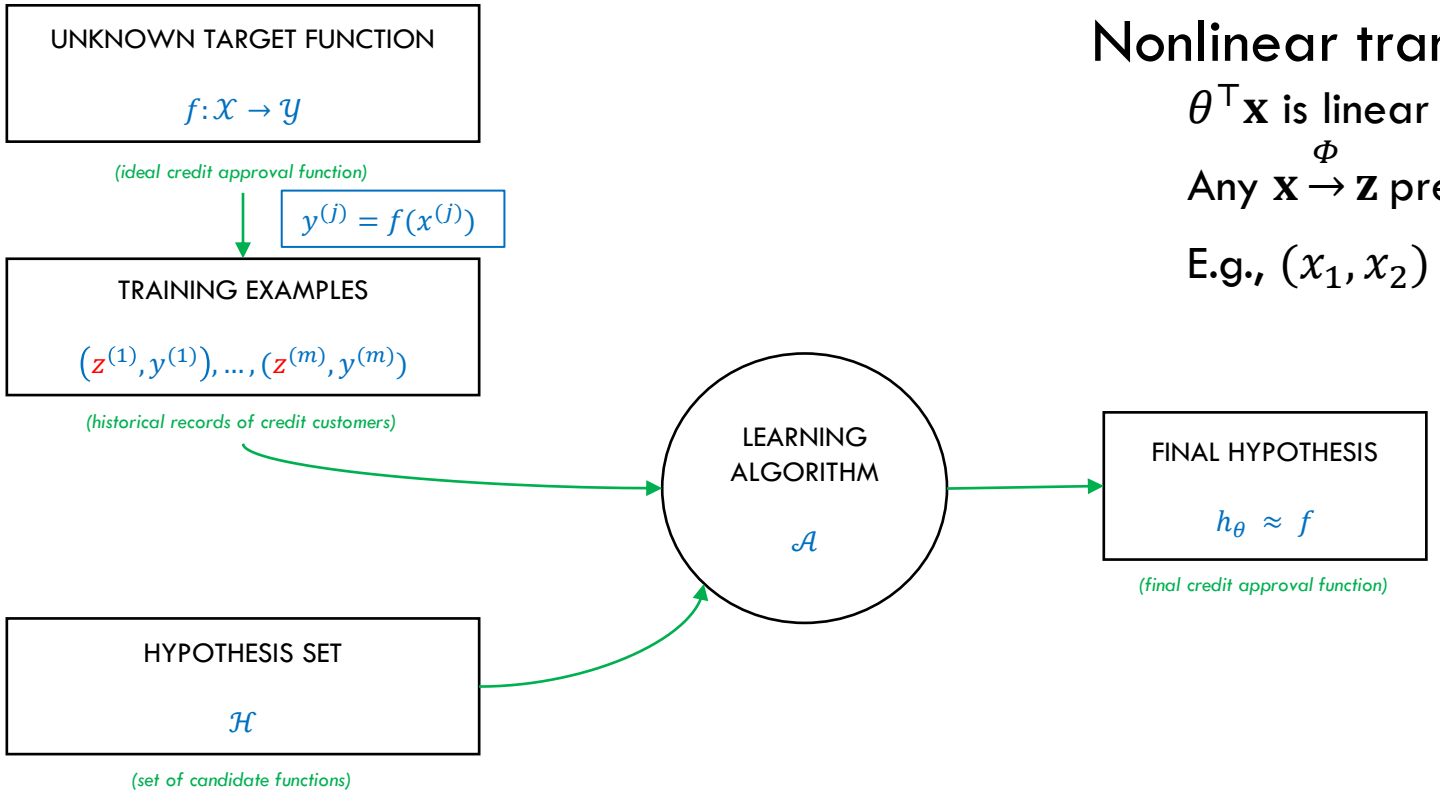
$$x^{(1)}, x^{(2)}, \dots, x^{(m)} \xrightarrow{\Phi} z^{(1)}, z^{(2)}, \dots, z^{(m)}$$

$$y^{(1)}, y^{(2)}, \dots, y^{(m)} \xrightarrow{\cancel{\Phi}} y^{(1)}, y^{(2)}, \dots, y^{(m)}$$

θ? No weights in $\mathcal{X}$ $\qquad \tilde{\theta} = (\theta_1, \theta_2, \dots, \theta_{\tilde{n}})$

$$\begin{aligned} h_{\tilde{\theta}}(\mathbf{x}) &= \operatorname{sign}\left(\tilde{\theta}^\top \mathbf{z}\right) \\ &= \operatorname{sign}(\tilde{\theta}^\top \Phi(\mathbf{x})) \end{aligned}$$

The learning diagram and summary



Nonlinear transformation

$\theta^\top \mathbf{x}$ is linear in θ

Any $\mathbf{x} \xrightarrow{\Phi} \mathbf{z}$ preserves this linearity

E.g., $(x_1, x_2) \xrightarrow{\Phi} (x_1^2, x_2^2)$

Normal Equation

for Linear Regression

CS3244 Machine Learning

NUS National University of Singapore
Department of Computer Science
School of Computing

The expression for J_{train}

$$J_{train} = \frac{1}{m}\sum_{j=1}^{m}(\theta^{\top}\mathbf{x}^{(j)} - y^{(j)})^2$$

$$= \frac{1}{m}\|\mathbf{X}\theta - y\|^2$$

where $\mathbf{X} = \begin{bmatrix} -\mathbf{x}^{(1)\top}- \\ -\mathbf{x}^{(2)\top}- \\ -\mathbf{x}^{(3)\top}- \\ \vdots \\ -\mathbf{x}^{(m)\top}- \end{bmatrix}, y = \begin{bmatrix} y^{(1)} \\ y^{(2)} \\ y^{(3)} \\ \vdots \\ y^{(m)} \end{bmatrix}$

Quick Question:
What are the dimensions of $\mathbf{X}$*?*

Minimizing J_{train}

$$J_{train} = \frac{1}{m} \|\mathbf{X}\theta - y\|^2$$

Minimizing J_{train}

$$J_{train} = \frac{1}{m}\|\mathbf{X}\theta - y\|^2$$

$$\nabla J_{train}(\theta) = \frac{2}{m}\mathbf{X}^\top(\mathbf{X}\theta - y) = \overleftarrow{0}$$

Column of zeroes

Minimizing J_{train}

$$J_{train} = \frac{1}{m}\|\mathbf{X}\theta - y\|^2$$

$$\nabla J_{train}(\theta) = \frac{2}{m}\mathbf{X}^\top(\mathbf{X}\theta - y) = \overleftarrow{0}$$

$$\boldsymbol{X}^\top\mathbf{X}\theta = \mathbf{X}^\top y$$

Minimizing J_{train}

$$J_{train} = \frac{1}{m}\|\mathbf{X}\theta - y\|^2$$

$$\nabla J_{train}(\theta) = \frac{2}{m}\mathbf{X}^\top(\mathbf{X}\theta - y) = \vec{0}$$

$$\underbrace{[\quad]}_{n+1}\underbrace{[\quad]}_{n+1} = \underbrace{[\quad]}_{n+1}\underbrace{[\quad]}_{1}^{n+1}$$

$$\boldsymbol{X}^\top\mathbf{X}\theta = \mathbf{X}^\top y$$

$$\theta = \mathbf{X}^\dagger y \qquad \text{where } \mathbf{X}^\dagger = (\mathbf{X}^\top\mathbf{X})^{-1}\mathbf{X}^\top$$

$\mathbf{X}^\dagger$ is the pseudo inverse of $\mathbf{X}$

The pseudo inverse $\mathbf{X}^{\dagger}$

$$\mathbf{X}^{\dagger} = (\mathbf{X}^{\top}\mathbf{X})^{-1}\mathbf{X}^{\top}$$

$$\underbrace{\begin{bmatrix} \quad \end{bmatrix}}_{n+1 \times m} \underbrace{\begin{bmatrix} \\ \\ \\ \end{bmatrix}}_{m \times n+1}$$

The pseudo inverse $\mathbf{X}^\dagger$

$$\mathbf{X}^\dagger = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top$$

$$\underbrace{\left[\underbrace{\left[\quad\right]}_{(n+1)\times(n+1)} \right]^{-1} \underbrace{\left[\quad\right]}_{(n+1)\times m}}_{(n+1)\times m}$$

Linear regression by Normal Equation

1. Construct the matrix $\mathbf{X}$ and the vector y from the data set as follows:

$$\mathbf{X} = \begin{bmatrix} -\mathbf{x}^{(1)\top}- \\ -\mathbf{x}^{(2)\top}- \\ -\mathbf{x}^{(3)\top}- \\ \vdots \\ -\mathbf{x}^{(m)\top}- \end{bmatrix}, y = \begin{bmatrix} y^{(1)} \\ y^{(2)} \\ y^{(3)} \\ \vdots \\ y^{(m)} \end{bmatrix}$$

input data matrix target vector

1. Compute the pseudo inverse $\mathbf{X}^{\dagger} = (\mathbf{X}^{\top}\mathbf{X})^{-1}\mathbf{X}^{\top}$
2. Return $\theta = \mathbf{X}^{\dagger}y$

The normal equation for LR. One-step learning!

Linear Regression – Summary

Two methods for training

1. Gradient Descent
   - Works well, even when n is large
   - Works even if $\mathbf{X}^\top\mathbf{X}$ is non-invertible

Your Turn: What are some reasons that $\mathbf{X}^\top\mathbf{X}$ might not be invertible?

2. Normal Equation
   - Don't need to choose α
   - Don't need to iterate
   - Need to compute $(\mathbf{X}^\top\mathbf{X})^{-1}$, an $O(n^3)$ operation

Using Regression for Classification?

CS3244 Machine Learning

NUS National University of Singapore
Department of Computer Science
School of Computing

Linear regression for classification

Linear regression learns a real-valued function $y = f(\mathbf{x}) \epsilon \mathbb{R}$

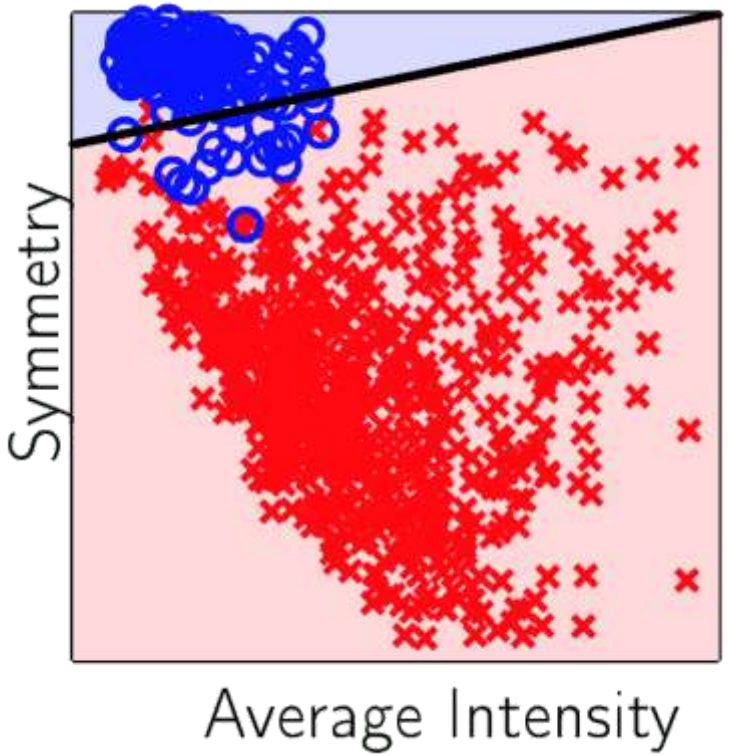
Binary valued functions are also real-valued! $\pm 1 \epsilon \mathbb{R}$

Use linear regression to get θ where $\theta^\top \mathbf{x}^{(j)} \approx y^{(j)} = \pm 1$

In this case, $\mathrm{sign}(\theta^\top \mathbf{x}^{(j)})$ is likely to agree with $y^{(j)} = \pm 1$

Good initial weights for classification

Why linear regression doesn't set good weights for classification



What's wrong with this picture?

Hint: think squared error

Stochastic Gradient Descent

CS3244 Machine Learning

NUS National University of Singapore | Department of Computer Science
School of Computing

Batch Gradient Descent

GD minimizes:

$$J_{train}(\theta) = \frac{1}{m}\sum_{j=1}^{m} \underbrace{j(h_\theta(x^{(j)}), y^{(j)})}_{\ln(1+e^{y^{(j)}\theta^\top x^{(j)}})}$$

By iterative steps along $-\nabla J$:

$$\Delta\theta = -\alpha \nabla J_{train}(\theta(t))$$

$-\alpha \nabla J_{train}$ is based on **all** examples $(\mathbf{X}, y)$

"batch" GD

SGD = PLA meets GD

A variation of GD that considers only the error on one data point.

Pick <u>one</u> $(\mathbf{x}^{(*)}, y^{(*)})$ at a time.

Apply GD to $j(h_\theta(\mathbf{x}^{(*)}), y^{(*)})$

"Average" Direction: $\mathbb{E}_*\left[-\nabla j(h_\theta(\mathbf{x}^{(*)}), y^{(*)})\right]$

$$= \frac{1}{m}\sum_{k=1}^{m} -\nabla j(h_\theta(\mathbf{x}^{(*)}), y^{(*)})$$
$$= -\nabla J_{train}$$

Stochastic Gradient Descent (SGD)

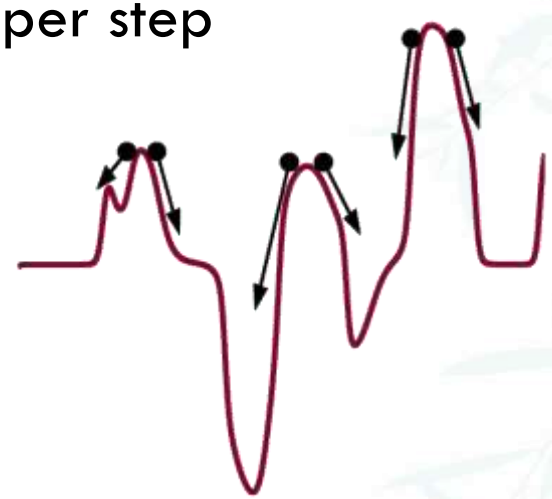
Photo credits: Wikipedia - Singapore Dollar

Benefits of SGD

1. Cheaper computation: Fraction $1/m$ cheaper per step

2. Stochastic: Helps escape local minima

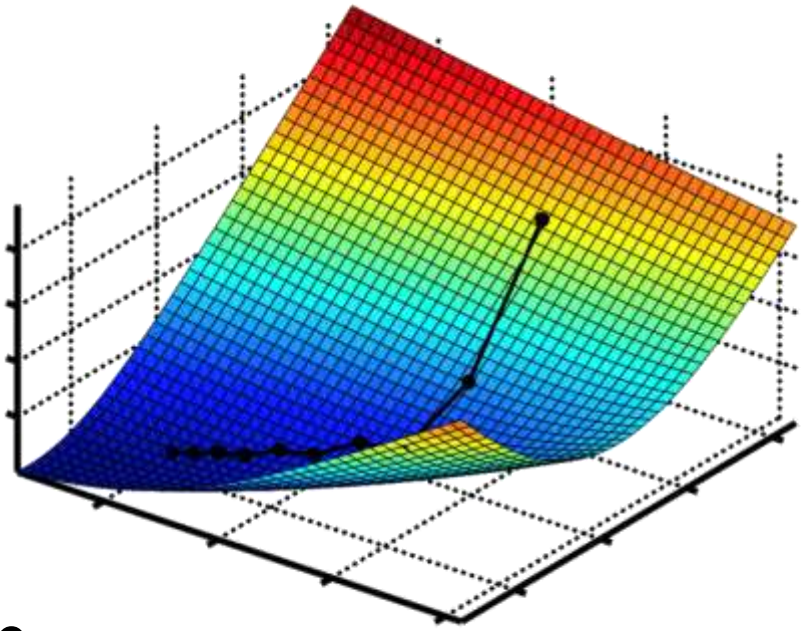
3. Simple



Rule of Thumb:
$\alpha = 0.1$ usually works!

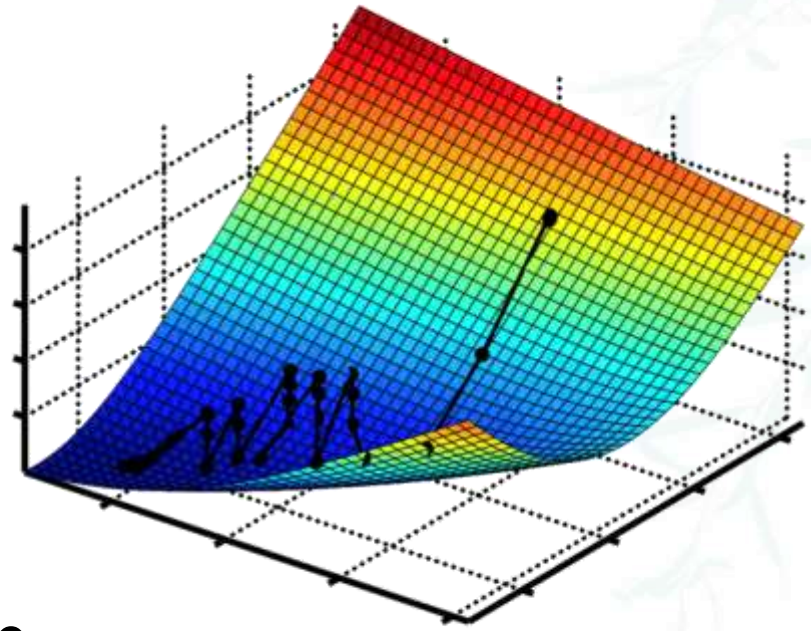
GD vs. SGD on $m = 10$

GD



10 steps

SGD



30 steps

Cross Entropy Cost Function

CS3244 Machine Learning

NUS National University of Singapore
Department of Computer Science
School of Computing

Logistic Regression Cost Function

For each $(\mathbf{x}, y)$, y is generated by probability $f(\mathbf{x})$

Plausible cost function based on likelihood:

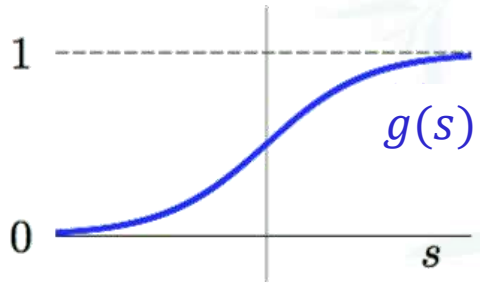
how "likely" is it to obtain output y from input $\mathbf{x}$?

If $h_\theta = f$,

$$P(y|x) = \begin{cases} h_\theta(\mathbf{x}) & \text{for } y = +1; \\ 1 - h_\theta(\mathbf{x}) & \text{for } y = -1. \end{cases}$$

Formula for likelihood

$$P(y|x) = \begin{cases} h_\theta(\mathbf{x}) & \text{for } y = +1; \\ 1 - h_\theta(\mathbf{x}) & \text{for } y = -1. \end{cases}$$



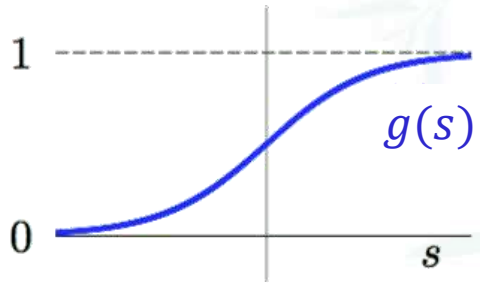
Substitute $h_\theta(\mathbf{x}) = g(\theta^\top \mathbf{x})$,

noting $g(-s) = 1 - g(s)$

$$P(y|x) = g(y \cdot \theta^\top \mathbf{x})$$

Formula for likelihood

$$P(y|x) = \begin{cases} h_\theta(\mathbf{x}) & \text{for } y = +1; \\ 1 - h_\theta(\mathbf{x}) & \text{for } y = -1. \end{cases}$$



Substitute $h_\theta(\mathbf{x}) = g(\theta^\top \mathbf{x})$,
noting $g(-s) = 1 - g(s)$

$$P(y|x) = g(y \cdot \theta^\top \mathbf{x})$$

Likelihood of $\mathbf{X} = (\mathbf{x}^{(1)}, y^{(1)}), \ldots, (\mathbf{x}^{(m)}, y^{(m)})$
where samples are i.i.d. is:

$$P(y^{(1)}, .., y^{(m)} | \mathbf{x}^{(1)}, .., \mathbf{x}^{(m)}) = \prod_{j=1}^{m} P(y^{(j)} | \mathbf{x}^{(j)}) \quad = \prod_{j=1}^{m} g(y^{(j)} \theta^\top \mathbf{x}^{(j)})$$

Maximizing the likelihood ≡ Minimizing cross entropy

$$= \max \quad \prod_{j=1}^{m} g\left(y^{(j)} \theta^{\top} x^{(j)}\right)$$

$$j(h_\theta(x^{(j)}), y^{(j)})$$

Maximizing the likelihood ≡ Minimizing cross entropy

$$= \ max \qquad \prod_{j=1}^{m} g\left(y^{(j)}\theta^{\top}x^{(j)}\right)$$

$$\Leftrightarrow max \qquad \frac{1}{\mathrm{m}}\ln\left(\prod_{j=1}^{m} g\left(y^{(j)}\theta^{\top}x^{(j)}\right)\right)$$

$$= \ max \qquad \frac{1}{m}\sum_{j=1}^{m}\ln g\left(y^{(j)}\theta^{\top}x^{(j)}\right)$$

$$\Leftrightarrow min \qquad -\frac{1}{m}\sum_{j=1}^{m}\ln g\left(y^{(j)}\theta^{\top}x^{(j)}\right)$$

$j(h_\theta(x^{(j)}), y^{(j)})$

Maximizing the likelihood ≡ Minimizing cross entropy

$$= \max \quad \prod_{j=1}^{m} g\left(y^{(j)}\theta^{\top}x^{(j)}\right)$$

$$\Leftrightarrow \max \quad \frac{1}{m}\ln\left(\prod_{j=1}^{m} g\left(y^{(j)}\theta^{\top}x^{(j)}\right)\right)$$

$$= \max \quad \frac{1}{m}\sum_{j=1}^{m} \ln g\left(y^{(j)}\theta^{\top}x^{(j)}\right)$$

$$\Leftrightarrow \min \quad -\frac{1}{m}\sum_{j=1}^{m} \ln g\left(y^{(j)}\theta^{\top}x^{(j)}\right)$$

$$= \min \quad \frac{1}{m}\sum_{j=1}^{m} \ln\left(\frac{1}{g\left(y^{(j)}\theta^{\top}x^{(j)}\right)}\right)$$

$$= \min \quad \frac{1}{m}\sum_{j=1}^{m} \ln\left(1+e^{-y^{(j)}\theta^{\top}x^{(j)}}\right)$$

$j(h_\theta(x^{(j)}), y^{(j)})$

Maximizing the likelihood ≡ Minimizing cross entropy

$$= \max \quad \prod_{j=1}^{m} g\left(y^{(j)}\theta^\top x^{(j)}\right)$$

$$\Leftrightarrow \max \quad \frac{1}{m}\ln\left(\prod_{j=1}^{m} g\left(y^{(j)}\theta^\top x^{(j)}\right)\right)$$

$$= \max \quad \frac{1}{m}\sum_{j=1}^{m} \ln g\left(y^{(j)}\theta^\top x^{(j)}\right)$$

$$\Leftrightarrow \min \quad -\frac{1}{m}\sum_{j=1}^{m} \ln g\left(y^{(j)}\theta^\top x^{(j)}\right)$$

$$= \min \quad \frac{1}{m}\sum_{j=1}^{m} \ln\left(\frac{1}{g\left(y^{(j)}\theta^\top x^{(j)}\right)}\right)$$

$$g(s) = \frac{e^s}{1+e^s} = \left[g(s) = \frac{1}{1+e^{-s}}\right]$$

Plug in

$$= \min \quad \frac{1}{m}\sum_{j=1}^{m} \ln\left(1+e^{-y^{(j)}\theta^\top x^{(j)}}\right)$$

$$\boxed{J_{train}(\theta) = \frac{1}{m}\sum_{j=1}^{m} \underbrace{\ln(1+e^{-y^{(j)}\theta^\top x^{(j)}})}_{j(h_\theta(x^{(j)}),\, y^{(j)})}}$$

"cross entropy" error

Minimizing J_{train}

For logistic regression,

$$J_{train}(\theta) = \frac{1}{m}\sum_{j=1}^{m} ln(1 + exp^{-y^{(j)}\theta^{\top}x^{(j)}})$$

Iterative Solution

Compare to linear regression:

$$J_{train}(\theta) = \frac{1}{m}\sum_{j=1}^{m}\left(\theta^{\top}x^{(j)} - y^{(j)}\right)^2$$

Closed-form Solution; Iteration also possible

Perceptron Convergence, Overview

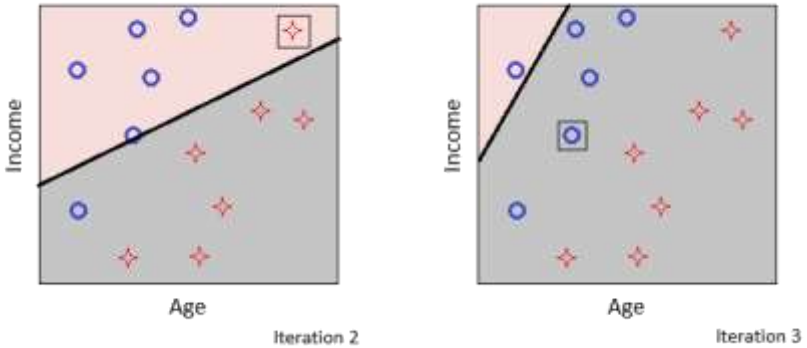
CS3244 Machine Learning

NUS National University of Singapore
Department of Computer Science
School of Computing

Perceptron Convergence

The weight update $\theta' \leftarrow \theta + y^{(i)}x^{(i)}$ tries to correct a single misclassified example.

But in doing so, sometimes the update can cause points that were previously correctly classified to become misclassified.



Is it possible that these weight updates will never terminate when the data is actually linearly separable?

Some Terminology

We'll need define some terminology to get through our proof, so:

To distinguish terms with respect to their values before or after a particular iteration, let's introduce $_(k)$ to mean the value of _ at iteration k and $_(k+1)$ to mean the value of _ at iteration $k+1$.

E.g., $x_{miss}(k) \equiv k^{\text{th}}$ iteration's misclassified example.

$||_||^2 \equiv$ the square of the norm of _.

Preliminaries

Originally our update is: $\theta(k+1) \leftarrow \theta(k) + y^{(i)}x^{(i)}$

- For a positive misclassification: $\theta(k+1) \leftarrow \theta(k) - x_{miss}(k)$
- For a negative misclassification: $\theta(k+1) \leftarrow \theta(k) + x_{miss}(k)$

Let's combine both classes into a single condition:
Multiply the negative training examples by its class label; i.e. -1.

Then, we can say a correct parameter setting θ^* satisfies:

$$\theta^* x^{(j)} > 0, \forall j \in [1 \dots m]$$

and our update equation is always:

$$\theta(k+1) \leftarrow \theta(k) + x_{miss}(k)$$

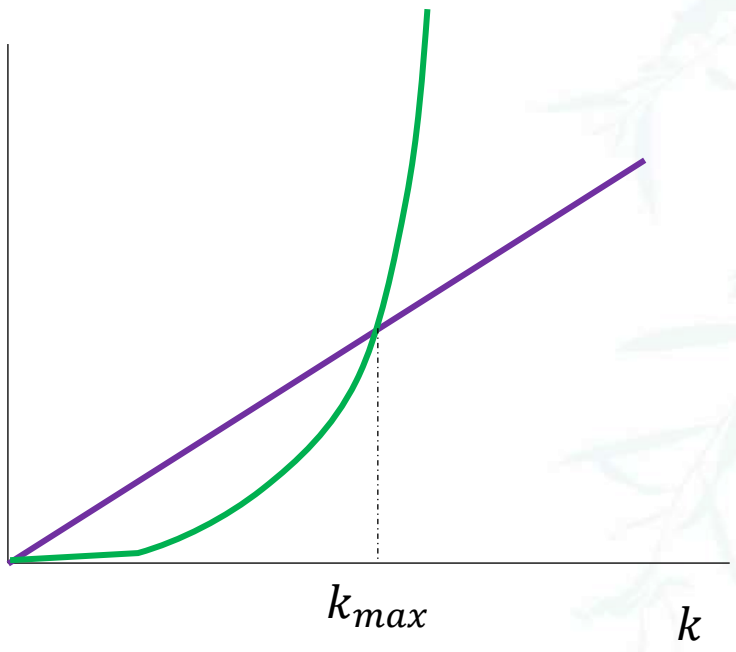
Outline: Proof by Contradiction

Goal: We'll prove that the updating process must stop after a finite number of iterations.

By contradiction, if PLA doesn't converge, there is always a misclassified example, and the weight update rule will be run forever: $\theta(k+1) \leftarrow \theta(k) + x_{miss}(k)$.

Line of argument for the proof

1. We'll find an upper, linear bound for how much $||\theta||$ can change from its initial value after k iterations.
2. We'll find a lower, quadratic bound for how much $||\theta||$ must change from its initial value after k iterations
3. These two bounds derives a finite upper bound for k.



Perceptron Convergence, Upper Bound

CS3244 Machine Learning

NUS National University of Singapore
Department of Computer Science
School of Computing

Linear Upper Bound

At iteration k+1, we have: $\theta(k+1) = \theta(k) + x_{miss}(k)$

Linear Upper Bound

At iteration k+1, we have: $\theta(k+1) = \theta(k) + x_{miss}(k)$

$$
\begin{aligned}
||\theta(k+1)||^2 &= ||\theta(k) + x_{miss}(k)||^2 \\
&= ||\theta(k)||^2 + ||x_{miss}(k)||^2 + 2\theta(k)^\top x_{miss}(k) \\
&\leq ||\theta(k)||^2 + ||x_{miss}(k)||^2
\end{aligned}
$$

Note: $\theta(k)^\top x_{miss}(k) < 0$ because x_{miss} was misclassified.

Linear Upper Bound

Recursively apply this idea:

$$||\theta(k)||^2 \leq ||\theta(k-1)||^2 + ||x_{miss}(k-1)||^2$$

Linear Upper Bound

Recursively apply this idea:

$$\left|\left|\theta(k)\right|\right|^2 \leq \left|\left|\theta(k-1)\right|\right|^2 + \left|\left|x_{miss}(k-1)\right|\right|^2$$

$$\leq \left|\left|\theta(k-2)\right|\right|^2 + \left|\left|x_{miss}(k-2)\right|\right|^2 + \left|\left|x_{miss}(k-1)\right|\right|^2$$

Linear Upper Bound

Recursively apply this idea:

$$
\begin{aligned}
||\theta(k)||^2 &\leq ||\theta(k-1)||^2 + ||x_{miss}(k-1)||^2 \\
&\leq ||\theta(k-2)||^2 + ||x_{miss}(k-2)||^2 + ||x_{miss}(k-1)||^2 \\
&\vdots \\
&\leq ||\theta(0)||^2 + \sum_{t=0}^{k-1} ||x_{miss}(t)||^2
\end{aligned}
$$

Linear Upper Bound

Recursively apply this idea:

$$
\begin{aligned}
||\theta(k)||^2 &\leq ||\theta(k-1)||^2 + ||x_{miss}(k-1)||^2 \\
&\leq ||\theta(k-2)||^2 + ||x_{miss}(k-2)||^2 + ||x_{miss}(k-1)||^2 \\
&\vdots \\
&\leq ||\theta(0)||^2 + \sum_{t=0}^{k-1} ||x_{miss}(t)||^2
\end{aligned}
$$

Without loss of generality, $||\theta(0)||^2 = 0$, if we initialize to zeroes.

Also define $M \equiv max\ ||x_{miss}(t)||^2$.

Linear Upper Bound

Recursively apply this idea:

$$
\begin{aligned}
||\theta(k)||^2 &\leq ||\theta(k-1)||^2 + ||x_{miss}(k-1)||^2 \\
&\leq ||\theta(k-2)||^2 + ||x_{miss}(k-2)||^2 + ||x_{miss}(k-1)||^2 \\
&\vdots \\
&\leq ||\theta(0)||^2 + \sum_{t=0}^{k-1} ||x_{miss}(t)||^2
\end{aligned}
$$

Without loss of generality, $||\theta(0)||^2 = 0$, if we initialize to zeroes.

Also define $M \equiv max\, ||x_{miss}(t)||^2$. Then,

$$
\begin{aligned}
||\theta(k)||^2 &\leq 0 + \sum_{t=0}^{k-1} M \\
&\leq kM.
\end{aligned}
$$

That is, the squared norm is upper bounded to grow linearly with each iteration.

Perceptron Convergence, Lower Bound

CS3244 Machine Learning

NUS National University of Singapore
Department of Computer Science
School of Computing

Quadratic Lower Bound

At iteration k, we have: $\theta(k) = \theta(k-1) + x_{miss}(k-1)$

Quadratic Lower Bound

At iteration k, we have: $\theta(k) = \theta(k-1) + x_{miss}(k-1)$

Expanding out:

$$\theta(k) = \theta(k-1) + x_{miss}(k-1)$$
$$= \theta(k-2) + x_{miss}(k-2) + x_{miss}(k-1)$$

Quadratic Lower Bound

At iteration k, we have: $\theta(k) = \theta(k-1) + x_{miss}(k-1)$

Expanding out:

$$\begin{aligned}\theta(k) &= \theta(k-1) + x_{miss}(k-1) \\ &= \theta(k-2) + x_{miss}(k-2) + x_{miss}(k-1) \\ &\vdots \\ &= \theta(0) + \sum_{i=0}^{k-1} x_{miss}(i)\end{aligned}$$

Quadratic Lower Bound

At iteration k, we have: $\theta(k) = \theta(k-1) + x_{miss}(k-1)$

Expanding out:

$$\begin{aligned}\theta(k) &= \theta(k-1) + x_{miss}(k-1) \\ &= \theta(k-2) + x_{miss}(k-2) + x_{miss}(k-1) \\ &\vdots \\ &= \theta(0) + \sum_{i=0}^{k-1} x_{miss}(i)\end{aligned}$$

Aside: There must exist a θ^* that separates both classes correctly. We can multiply the above by θ^*. Expanding out:

$$\theta(k)^\top \theta^* = 0 + \sum_{i=0}^{k-1} x_{miss}(i)^\top \theta^*$$

Quadratic Lower Bound

At iteration k, we have: $\theta(k) = \theta(k-1) + x_{miss}(k-1)$

Expanding out:

$$\begin{aligned}\theta(k) &= \theta(k-1) + x_{miss}(k-1) \\ &= \theta(k-2) + x_{miss}(k-2) + x_{miss}(k-1) \\ &\vdots \\ &= \theta(0) + \sum_{i=0}^{k-1} x_{miss}(i)\end{aligned}$$

Aside: There must exist a θ^* that separates both classes correctly. We can multiply the above by θ^*. Expanding out:

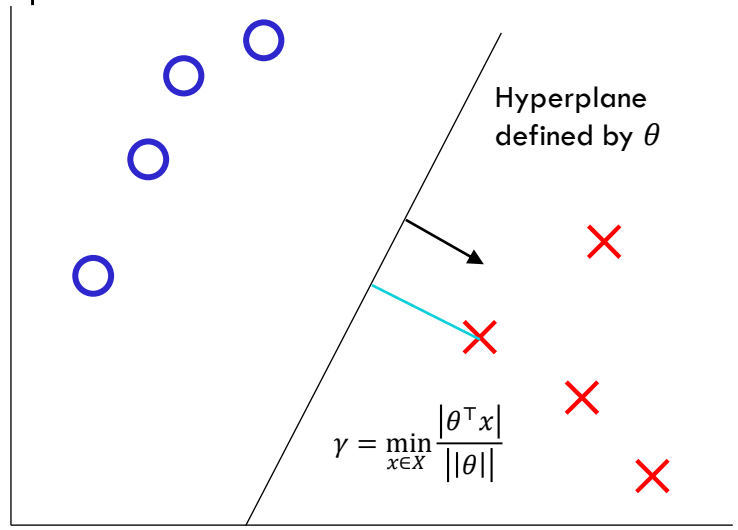
$$\theta(k)^\top \theta^* = 0 + \sum_{i=0}^{k-1} x_{miss}(i)^\top \theta^*$$

Let's define a quantity that will help with a lower bound.

Define $\gamma = \min_{x \in X} \left|\theta^\top x_{miss}(i)\right|$

Aside: γ, the (L2) margin

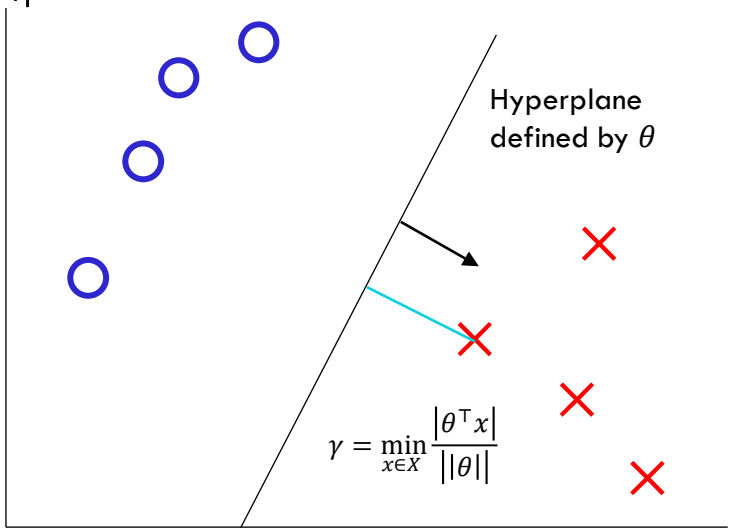
$\frac{|\theta^\top x|}{||\theta||}$ = distance between separating hyperplane and x.



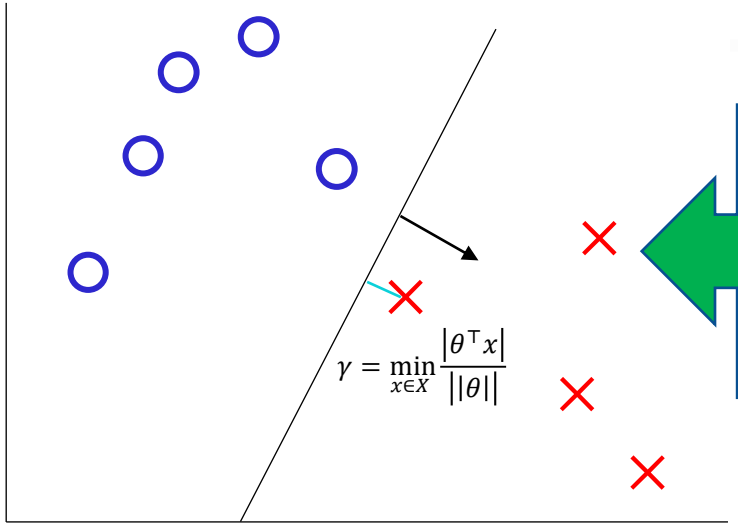
Larger Margin γ
(more wiggle room)

Aside: γ, the (L2) margin

$\frac{|\theta^\top x|}{||\theta||}$ = distance between separating hyperplane and x.



Larger Margin γ
(more wiggle room)



Smaller Margin γ
(less wiggle room)

Sneak peek at Support Vector Machines (SVMs)

Quadratic Lower Bound

Aside: There must exist a θ^* that separates both classes correctly. We can multiply the above by θ^*. Expanding out:

$$\theta(k)^\top \theta^* = 0 + \sum_{i=0}^{k-1} x_{miss}(i)^\top \theta^*$$

Let's define a quantity that will help with a lower bound.

Define $\gamma = \min_{x \in X} \left| \theta^\top x_{miss}(i) \right|$

Cauchy–Schwarz inequality

From Wikipedia, the free encyclopedia

In mathematics, the **Cauchy–Schwarz inequality**, also known as the **Cauchy–Bunyakovsky–Schwarz inequality**, is a useful inequality encountered in many different settings, such as linear algebra, analysis, probability theory, vector algebra and other areas. It is considered to be one of the most important inequalities in all of mathematics.[1]

The inequality for sums was published by Augustin-Louis Cauchy (1821), while the corresponding inequality for integrals was first proved by Viktor Bunyakovsky (1859). The modern proof of the integral inequality was given by Hermann Amandus Schwarz (1888).[1]

Statement of the inequality [edit]

The Cauchy–Schwarz inequality states that for all vectors u and v of an inner product space it is true that

$$|\langle \mathbf{u}, \mathbf{v}\rangle|^2 \le \langle \mathbf{u}, \mathbf{u}\rangle \cdot \langle \mathbf{v}, \mathbf{v}\rangle,$$

where $\langle \cdot, \cdot \rangle$ is the inner product. Examples of inner products include the real and complex dot product, see the examples in inner product. Equivalently, by taking the square root of both sides, and referring to the norms of the vectors, the inequality is written as[2][3]

$$|\langle \mathbf{u}, \mathbf{v}\rangle| \le \|\mathbf{u}\|\|\mathbf{v}\|.$$

Moreover, the two sides are equal if and only if $\mathbf{u}$ and $\mathbf{v}$ are linearly dependent (meaning they are parallel: one of the vector's magnitudes is zero, or one is a scalar multiple of the other).[4][5]

If $u_1, \ldots, u_n \in \mathbb{C}$ and $v_1, \ldots, v_n \in \mathbb{C}$, and the inner product is the standard complex inner product, then the inequality may be restated more explicitly as follows (where the bar notation is used for complex conjugation):

$$|u_1\bar{v}_1 + \cdots + u_n\bar{v}_n|^2 \le (|u_1|^2 + \cdots + |u_n|^2)(|v_1|^2 + \cdots + |v_n|^2)$$

or

$$\left|\sum_{i=1}^{n} u_i\bar{v}_i\right|^2 \le \sum_{j=1}^{n} |u_j|^2 \sum_{k=1}^{n} |v_k|^2.$$

Proofs [edit]

First proof [edit]

Let u and v be arbitrary vectors in a vector space over $\mathbb{F}$ with an inner product, where $\mathbb{F}$ is the field of real or complex numbers. We prove the

Quadratic Lower Bound

Aside: There must exist a θ^* that separates both classes correctly. We can multiply the above by θ^*.
Expanding out:

$$\theta(k)^\top \theta^* = 0 + \sum_{i=0}^{k-1} x_{miss}(i)^\top \theta^*$$

Let's define a quantity that will help with a lower bound.

Define $\gamma = \min_{x \in X} \left|\theta^\top x_{miss}(i)\right|$

$$k\gamma \leq \theta(k)^\top \theta^*$$

$$k^2\gamma^2 \leq \left|\left|\theta(k)^\top \theta^*\right|\right|^2$$ (Squaring both sides; as they are both scalars)

$$\leq \left|\left|\theta(k)\right|\right|^2 \left|\left|\theta^*\right|\right|^2$$ (By the Cauchy-Schwartz inequality)

$$\leq kM\left|\left|\theta^*\right|\right|^2$$ (from our **upper bound**)

Putting the bounds together

$$k^2\gamma^2 \leq kM\left|\left|\theta^*\right|\right|^2$$

$$k \leq \frac{M\left|\left|\theta^*\right|\right|^2}{\gamma^2}$$

$\therefore$ That is, k can iterate up to a max of $\frac{M\left|\left|\theta^*\right|\right|^2}{\gamma^2}$ times. ■

Note that $k \propto \frac{1}{\gamma^2}$ (where $\gamma < 1$). It takes exponentially more iterations to find a satisficing separator, given a smaller margin.